

Algorytmy i Struktury Danych II

Dokumentacja Projektowa

Mateusz Górniak, Michał Jeziorski, Kuba Kaliski, Daniel Walendowski

0. Przyjęte założenia projektowe

Królestwo położone jest na kartezjańskim układzie współrzędnych, gdzie na podstawie 3 parametrów: rozmiaru, proporcji i perkolacji losujemy rozstawienie domków i kopalni na tym układzie.

W jednym domku znajduje się jeden krasnal, który posiada swój preferowany surowiec do wydobywania. Wydobywanie ulubionego surowca jest 5x bardziej wydajne niż jakiegokolwiek innego surowca. Następnie mamy kopalnie – każda posiada surowiec, który jest w niej wydobywany, oraz określoną pojemność (losowa wartość z przedziału $[1, 4]$).

1. Otoczka Wypukła

Zadanie polega na znalezieniu najkrótszej ścieżki otaczającej wszystkie kopalnie, czyli wyznaczeniu otoczki wypukłej. W projekcie zastosowaliśmy algorytm łańcucha monotonicznego (ang. *Monotone Chain Algorithm*, znany również jako algorytm Andrewsa).

Złożoność czasowa: $O(n \log n)$
--

1.1 Uzasadnienie wyboru algorytmu

W porównaniu do innych algorytmów wyznaczania otoczki wypukłej, np. algorytmu Grahama, algorytm łańcucha monotonicznego wykazuje lepszą stabilność numeryczną. Algorytm Grahama wymaga znalezienia najniższego punktu i posortowania pozostałych punktów według kąta biegunowego względem tego punktu. Sortowanie kątowe wymaga oddzielnych funkcji dla obliczeń trygonometrycznych, przez co może być podatne na błędy numeryczne, zwłaszcza przy liczbach zmiennoprzecinkowych, gdzie zaokrąglenia mogą prowadzić do nieprawidłowych wyników.

Natomiast algorytm łańcucha monotonicznego sortuje rosnąco punkty według współrzędnych kartezjańskich (najpierw po współrzędnej x w przypadku remisu po y) (od skrajnego lewego punktu), a do sprawdzania skrętów trasy używa wyłącznie operacji mnożenia i odejmowania (iloczyn wektorowy). Eliminuje to problemy z precyzją, które pojawiają się przy obliczeniach trygonometrycznych.

1.2 Opis algorytmu

Algorytm składa się z czterech głównych etapów:

Krok 1 — Sortowanie punktów

Punkty (kopalnie) sortowane są rosnąco według współrzędnej X . W przypadku dwóch punktów o tej samej współrzędnej X , jako kryterium pomocnicze stosowana jest współrzędna Y .

Krok 2 — Budowa dolnej otoczki

Algorytm iteruje po posortowanej liście punktów od lewej do prawej, dodając kolejne punkty do stosu. Jeśli trzy ostatnie punkty na stosie tworzą wklęsłość (skręt w lewo), środkowy z nich jest usuwany. Operacja sprawdzania kierunku skrętu opiera się na iloczynie wektorowym.

Krok 3 — Budowa górnej otoczki

Lista punktów jest przetwarzana w identyczny sposób jak przy budowie dolnej otoczki, lecz w odwrotnej kolejności — od prawej do lewej (od końca posortowanej listy).

Krok 4 — Łączenie otoczek

Dolna i górna część otoczki są łączone w jedną zamkniętą ścieżkę. Podczas łączenia pomijane są powtarzające się punkty skrajne, aby uniknąć duplikatów na liście wynikowej.

2. Problem znalezienia najgłośniejszego dekametrowca na przedziale

Problem polega na efektywnym wyznaczaniu maksymalnej głośności na zadanym przedziale (zapytanie) dekametrowców, którzy są rozstawieni na trasie otoczki wypukłej oraz posiadają swoją wartość głośności.

2.1 Drzewo Przedziałowe (Segment Tree)

Drzewo przedziałowe oferuje czas zapytania $O(\log n)$ oraz czas budowy $O(n)$. Kluczową zaletą tej struktury jest efektywna obsługa dużej ilości zapytań oraz możliwość dynamicznej zmiany danych - zmiana dowolnej wartości wymaga zaktualizowania odpowiadającego jej liścia oraz przeliczenia ścieżki prowadzącej do korzenia drzewa, co kosztuje $O(\log n)$.

2.2 Porównanie i uzasadnienie wyboru

Można zadać pytanie, czy w tym przypadku nie lepiej użyć tablicy rzadkiej, która oferuje stały czas zapytania $O(1)$ oraz dłuższy czas budowy $O(n \log n)$. W teorii zastosowanie tablicy rzadkiej nie jest złym pomysłem, natomiast przy tak krytycznym systemie związanym z obroną granicy królestwa każda milisekunda się liczy, a na naszej granicy może zostać rozstawiona ogromna liczba dekametrowców. Drzewo przedziałowe zyskuje przewagę w czasie z każdym dodatkowym zapytaniem.

Kryterium	Tablica Rzadka	Drzewo Przedziałowe
Czas zapytania	$O(1)$	$O(\log n)$
Czas modyfikacji	$O(n \log n)$	$O(\log n)$
Czas budowy struktury	$O(n \log n)$	$O(n)$

2.3 Sposób rozstawienia dekametrowców

Nasi dekametrowcy rozstawieni są na trasie księcia w równych odstępach co 15 jednostek. Rozstawienie odbywa się za pomocą algorytmu iteracyjnego próbkowania. Rozstawianie zaczyna się od pierwszego wierzchołka otoczki wypukłej (skrajny lewy punkt) następnie algorytm iteruje po wszystkich krawędziach obliczając dystans na krawędzi. Algorytm pilnuje reszty z dystansu, jeśli do końca danej krawędzi został dystans mniejszy niż 15 jednostek algorytm zapamięta ten dystans i rozpoczynając operacje na następnej krawędzi uwzględni on to przeniesienie.

3. Zapisanie wiedzy oszczędzając elektroniczny papier

Archiwizacją danych w taki sposób, by zajmowały jak najmniej miejsca, zajął się program „Archiwizator”, który został zbudowany w oparciu o kodowanie Huffmana. Program najpierw generuje kody, następnie koduje tekst. Zakodowany tekst zapisuje do pliku .huff, a kody do pliku .huffcode. Następnie z plików .huff i .huffcode można wygenerować pierwotny plik.

Złożoność obliczeniowa algorytmu to $O(n \log n)$.

3.1 Zasada działania algorytmu

Algorytm polega na zbudowaniu drzewa kodującego dla liter znajdujących się w tekście do zakodowania. Drzewo budowane jest na podstawie liczby wystąpień liter w tekście – najczęściej występująca litera otrzymuje najkrótszy kod, a każda gałąź drzewa jest oznaczona zerem lub jedynką. Oznaczenia przy gałęziach pokazują, w jaki sposób należy kodować litery w tekście – najczęściej występująca litera otrzyma kod 0, zajmując w ten sposób najmniej miejsca, najrzadziej występujące litery dostaną dłuższe kodowanie i będą zajmować najwięcej miejsca. Dzięki takiemu sposobowi zapisania zakodowanego tekstu można znacząco zmniejszyć jego rozmiar – typowa oszczędność wynosi 20–50%. Im większa nierówność w częstotliwości występowania poszczególnych znaków, tym większa oszczędność.

3.2 Skuteczność

W celu sprawdzenia praktycznej skuteczności i szybkości działania zostały wygenerowane trzy rodzaje plików – pierwszy z zerową entropią, z jednym rodzajem znaku – „a”, drugi z przykładowymi danymi użytymi do budowy wioski krasnoludków, trzeci z dużą entropią, z pseudolosowymi znakami ASCII z zakresu 32–126. Każdy z trzech rodzajów pliku został wygenerowany w czterech wersjach w zależności od liczby znaków w nich się znajdujących: „mikro dane” – 5 658 znaków, „małe dane” – 240 224 znaków, „średnie dane” – 2 714 422 znaków i „duże dane” – 13 925 781 znaków.

W przypadku plików z przykładowymi danymi z projektu oraz danych o dużej entropii skuteczność kompresji pozostawała podobna bez względu na rozmiar danych i oszczędność oscylowała w okolicach kolejno 47% i 17%. Dla danych z jednym znakiem zakodowany plik za każdym razem ważył 1 bajt, co w przypadku „dużych danych” przekłada się na oszczędność 99,999993%. Oczywiście należy uwzględnić, że jest to wyjątkowo skrajny przypadek.

Czasy kompresji dla wszystkich rodzajów danych wynosiły te same rzędy wielkości – dla „mikro danych” były to tysięczne sekundy, dla „małych danych” setne sekundy, dla „średnich danych” dziesiętne sekundy, natomiast dla „dużych danych” już całe sekundy. Dane z zerową

entropią kompresowały się najszybciej, najwolniejsza kompresja następowała dla danych o dużej entropii. Czasy dekompresji dla danych o zerowej entropii nie różniły się znacząco w zależności od wielkości danych i wynosiły około 0,001 s. Dla pozostałych danych wraz z rozmiarem zwiększały się czasy dekompresji – bardziej niż w przypadku kompresji – dla „dużych danych” było to już kilkaset sekund. Różnica w czasie działania dekompresji dla poszczególnych rodzajów danych była zauważalna. W celu dokładniejszego zapoznania się z wynikami pomiarów proszę zapoznać się z plikiem pomiary_archiwizatora.txt.

4. Wyszukiwanie potrzebnych informacji dla potomnych

Problematyka polega na znalezieniu szukanego słowa w czeluściach archiwum krasnoludków, czyli na optymalnym wyszukiwaniu podanej frazy w pliku. W narzędziu „wyszukiwarka” zastosowany został algorytm wyszukiwania wzorca Boyer’a i Moore’a, który pozwala na znalezienie w tekście pierwszego wystąpienia wzorca.

W pesymistycznym przypadku złożoność czasowa algorytmu to $O(nm)$.

4.1 Uzasadnienie i działanie

Wybór algorytmu Boyer’a i Moore’a podyktowany został szybkością działania. W przeciwieństwie do algorytmu naiwnego nie porównuje znaków od początku wzorca przy każdym przesunięciu, tylko wykorzystuje informacje o niepasującym znaku, dzięki czemu pomija fragmenty tekstu. W porównaniu z Rabin-Karpem Boyer-Moore nie opiera się na haszowaniu, dzięki czemu unika problemów z kolizjami i dodatkowej pracy związanej z obliczaniem oraz weryfikacją wartości hasza. Natomiast względem Knutha-Morrisa-Pratta Boyer-Moore potrafi wykonywać większe przesunięcia wzorca niż KMP i potrzebuje mniej porównań.

W algorytmie dużą rolę odgrywają heurystyki – heurystyka złego znaku oraz heurystyka dobrego sufiksu. Heurystyka złego znaku polega na tym, że po znalezieniu niezgodności wzorzec przesuwa się tak, aby niepasujący znak tekstu został ustawiony pod ostatnim wystąpieniem tego samego znaku we wzorcu, a jeśli takie nie występuje, wzorzec przesuwa się jeszcze dalej. Heurystyka dobrego sufiksu wykorzystuje dopasowany już końcowy fragment wzorca i przesuwa go tak, by ten sam fragment pojawił się w innym miejscu wzorca albo by dopasować jego najdłuższy możliwy prefiks. Gotowe narzędzie „wyszukiwarka” wyszukuje nie pierwsze wystąpienie, a wszystkie wystąpienia frazy w podanym pliku.

4.2 Pomiary

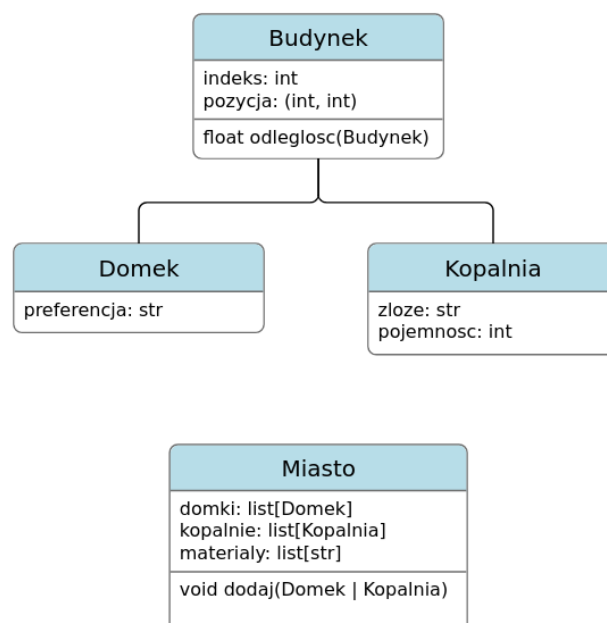
W ramach niniejszej dokumentacji przeprowadzone zostały testy na tekstach zawierających 1 000 000 znaków w celu sprawdzenia praktycznej wydajności. W tym celu przygotowanych zostało 11 plików. Pierwszy zawierał jedno wystąpienie dwuliterowej frazy na początku tekstu, drugi zawierał dwuliterową frazę na końcu tekstu. Średnie uzyskiwane czasy wynosiły kolejno 0,1781 s oraz 0,1889 s. Rozmieszczenie wzorca w tekście nie wpływa na czas wyszukiwania tak znacząco, jak długość wzorca. Analogiczne wyszukiwanie z czteroliterowym wzorcem średnio dało czasy 0,0931 s i 0,0971 s. Następnie zostały przeprowadzone testy na dwóch wystąpieniach wzorca – średnio 0,2223 s dla dwuliterowego wzorca i 0,1017 s dla czteroliterowego wzorca – oraz pięciu wystąpieniach – średnio 0,1769 s dla dwuliterowego

wzorca oraz 0,1007 s dla czteroliterowego wzorca. W ostatnich trzech plikach umieszczonych zostało 100 szukanych wzorców w trzech wariantach – wzorce dwuliterowe, wzorce czteroliterowe oraz wzorce czteroliterowe, podczas gdy reszta tekstu to pierwsze dwie litery wzorca. Wyszukiwanie trwało średnio odpowiednio 0,2403 s, 0,1275 s oraz 0,2225 s. Powyższe testy wskazują, że największe znaczenie dla wydajności narzędzia ma długość frazy – im dłuższa, tym krócej trwa wyszukiwanie. W celu dokładniejszego zapoznania się z wynikami pomiarów proszę zapoznać się z plikiem pomiary_wyszukiwarki.txt.

5. Problem przydziału pracy

Posiadamy pewien zbiór domków, w których mieszkają krasnoludki oraz kopalnie. W kopalniach można znaleźć surowce, a krasnoludki mają swój jeden ulubiony surowiec do kopania, który wydobywają najlepiej. Każdy inny surowiec wydobywają z pięciokrotnie zmniejszoną wydajnością. Budynki znajdują się na płaszczyźnie o współrzędnych kartezjańskich, a naszym zadaniem jest znalezienie maksymalnej ilości par domek-kopalnia, takich by łączna droga między nimi była najmniejsza, a ilość wydobytych surowców największa.

Do zapisania informacji o budynkach wykorzystujemy bazową klasę Budynek, która posiada pola indeksu – unikalnej liczby całkowitej do identyfikowania budynku, pozycji – krotki ze współzrędnymi (x, y), oraz na potrzeby wizualizacji listę Sąsiadów (jego indeks i odległość). Klasa Domek dziedziczy z Budynku, dodając pole preferencja z ulubionym surowcem krasnoludka w nim mieszkającego. Przyjmujemy, że e jednym domku mieszka jeden krasnoludek. Klasa Kopalnia dziedziczy z Budynku rozszerzając ją o pola złożo – możliwy do wydobywania surowiec i pojemność – ile maksymalnie krasnoludków może być przypisanych do kopalni. Pomocnicza klasa Miasto trzyma listę domków i kopalni wraz z listą materiałów, które są dostępne w kopalniach.



5.1 Wybór par

Problem możemy przekształcić w znalezienie największego skojarzenia grafu dwudzielnego, gdzie dwa zbiory wierzchołków tworzą domki i kopalnie. Chcemy żeby krawędzie należące do skojarzenia miały możliwie jak najmniejszy koszt, gdzie za koszt przyjmujemy jednostkę odległości na wydobyty surowiec. Dla preferowanych przez krasnoludka surowców kosztem będzie odległość, a dla innych, jako że wydajność wydobywania spada pięciokrotnie, kosztem będzie pięciokrotność odległości.

Do znalezienia największego skojarzenia grafu, stowarzyszymy go z siecią przepływową, dodając źródło połączone ze wszystkimi domkami, oraz ujście połączone ze wszystkimi kopalniami. Przepustowość krawędzi źródło-domek oraz domek-kopalnia będzie wynosić 1, natomiast kopalnia-ujście będzie wynosić pojemność kopalni.

Jako że do kopalni może być przypisanych więcej niż jeden krasnoludek, możemy wyobrazić sobie n kopii tej samej kopalni o pojemności 1, gdzie n jest jej oryginalną pojemnością. Wtedy poprawna strategia szukania skojarzenia grafu da nam poprawny wynik, znajdując największą ilość par domek-kopalnia (według rozwiązania na wykładzie). Możemy jednak skompresować w grafie n identycznych kopalni (o tym samym surowcu, pozycji i indeksie) do jednej o przepustowości wyjściowej równej n . Każdy domek może połączyć się tylko z jedną kopalnią, więc przepustowość między domkiem a zbiorem kopii jednej kopalni będzie równa dalej 1, ale do każdej kopii może połączyć się n domków, czyli przepływ wypływający ze zbioru kopii do ujścia może być równy maksymalnie n (n kopii razy przepustowość 1). Algorytm największego przepływu w naszej zmodyfikowanej sieci przepływowej da nam wynik analogiczny do znalezienia skojarzenia w grafie, gdzie kopalnie zostają rozszczipione na osobne kopie o jednym miejscu. Wynik jednak nie będzie skojarzeniem, jako że do jednej kopalni może zostać przypisanych więcej niż jeden domek, lecz jest to oczekiwany skutek.

Użyjemy do tego metody Edmondsa-Karpa i algorytmu Cycle Canceling Adnrew V. Goldberg'a i Roberta E. Trajan'a z 1989.

5.2 Znajdowanie maksymalnego przepływu

Klasa `SiecPrzeplywowa` generuje instancję klasy `GrafDwudzielny`, która z informacji z obiektu `Miasto` tworzy listę wierzchołków połączonych z innymi. Lista wierzchołków reprezentuje skierowany graf ważony, gdzie krawędzie przepływają od źródła, przez domki do kopalni i na końcu do ujścia, w tym kierunku. Wagi krawędzi to koszt liczony w zależności od odległości i dopasowania materiału do preferencji. Po wygenerowaniu grafu stworzona jest sieć przepływowa, uzupełniając graf o krawędzie w przeciwnym kierunku i o wartości przepustowości krawędzi. Klasa `SiecPrzeplywowa` odpowiada za obliczenie wyników algorytmów na wytworzonej sieci przepływowej. Do znalezienia maksymalnego przepływu wykorzystujemy algorytm Edmondsa-Karpa z wybieraniem ścieżki powiększającej przez algorytm przeszukiwania wszerz.

Algorytm przeszukiwania wszerz (BFS) znajduje najkrótszą drogę od źródła do ujścia przez sieć przepływową, po czym na tej trasie zostaje przesłana maksymalna możliwa ilość przepływu. Takie modyfikacje wykonują się dopóki BFS jest w stanie znaleźć drogę do ujścia, gdy nie znajdzie żadnej ścieżki oznacza to, że maksymalny przepływ zostaje znaleziony.

Pesymistyczna złożoność czasowa algorytmu Edmondsa-Karpa to $O(|V||E|^2)$.

By skrócić czas wykonywania algorytmu, liczony jest początkowy przepływ. Algorytm przechodzi przez wszystkie domki szukając pierwszej nie zapełnionej kopalni z

którą zgadzają się preferencją, zmieniając przy tym przepływ w sieci. Ten naiwny zabieg nie gwarantuje nam największego przepływu, lecz skraca czas wykonywania algorytmu zmniejszając potrzebną ilość wyliczeń ścieżki powiększającej.

Algorytm BFS ma złożoność $O(|V|+|E|)$, u nas ilość wierzchołków to ilość domków i kopalni, a krawędzią połączona jest każda para, czyli złożoność wynosi $O(|D|+|K|+|D||K|)$. Minimalna liczba znalezionych połączeń wynosi tyle ile budynków o mniejszej liczności. Załóżmy, że jest mniej kopalni (choć działa to także w drugą stronę). Każda kopalnia może przyjąć conajmniej jeden domek, więc algorytm szukając największej ilości połączeń powinien znaleźć conajmniej $|K|$ połączeń. To znaczy, że algorytm Edmondsa-Karpa musi conajmniej $|K|$ razy wykonać algorytm złożony $O(|D|+|K|+|D||K|)$. Nasz zabieg ma pesymistyczną złożoność $O(|D||K|)$ (jako że musi przejść po każdej parze domek-kopalnia) i musi wykonać się 1 raz. Potem kolejne przejścia BFS będą polepszać wynik w obu przypadkach. W najgorszym przypadku nasz zabieg nie polepszy sytuacji, znajdując same niepotrzebne połączenia i algorytm Edmondsa-Karpa będzie musiał wykonać pracę bez początkowej ulgi. Jest to jednak mało prawdopodobny przypadek, a w testach wykazuje poprawę prędkości.

Średnia czasu ze 100 testów dla 25 budynków w proporcji ok. 70% domków: oryginalny algorytm zajmuje 0.0109s, a z naszym usprawnieniem 0.0093s.

Średnia czasu ze 100 testów dla 100 budynków w powyższej proporcji: oryginalny algorytm zajmuje 3.0255s, z naszym usprawnieniem 3.4590s.

Poprawność algorytmu jest gwarantowana przez poprawność algorytmu Edmondsa-Karpa (z wykładu). Nasza modyfikacja nie wpływa na poprawność, jako że generuje poprawną sieć przepływową, nawet jeżeli całkowity przepływ nie zostaje polepszony.

5.3 Usuwanie ujemnych cykli

Po znalezieniu maksymalnego przepływu, wykorzystujemy algorytm Cycle Canceling, który polega na znajdowaniu ujemnych cykli w sieci przepływowej i przesyłania przez taki cykl przepływu póki nie zniknie. Do znajdowania ujemnych cykli wykorzystujemy algorytm Bellmana-Forda-Moore'a.

Algorytm Bellmana-Forda polega na wielokrotnej relaksacji, czyli zmniejszaniu odległości do źródła, sprawdzając czy nie ma w sąsiedztwie takiego wierzchołka, z którego trasa byłaby krótsza niż nasza aktualna. Ten algorytm znajduje najkrótszą drogę w grafie po $|V|-1$ krokach relaksacji, lecz pozwala nam na znalezienie ujemnych cykli, jeżeli sprawdzimy czy po jeszcze jednym kroku relaksacji nie zmienia się odległości. Te wierzchołki którym odległość się zmieni, należą do ujemnego cyklu, bądź z takiego cyklu wychodzą. Algorytm przechowuje także listę rodziców, czyli wierzchołków które doprowadzą nas do źródła o ile nie ma cykli ujemnych. Jeśli ujemny cykl istnieje, możemy wrócić się po tych rodzicach by znaleźć ten cykl. Modyfikacja tego algorytmu nazwana algorytmem Bellmana-Forda-Moore'a polega na zauważeniu, że jeżeli po kroku relaksacji najkrótsza odległość się nie zmienia, to nie trzeba relaksować takiego wierzchołka poraz kolejny. W naszym kodzie przechowujemy listę wierzchołków do relaksacji. Jeżeli wierzchołek został zrelaksowany, zostaje usunięty z listy, a jeżeli w kroku relaksacji jego odległość się zmieniła, zostaje dodany z powrotem. Dowód poprawności tego algorytmu można przeczytać z wykładu "MATH 409 LECTURE 11 THE MOORE-BELLMAN-FORD ALGORITHM" Uniwersytetu Waszyngtońskiego (https://sites.math.washington.edu/~thomas/teaching/m409_s2010_web/lecture11.pdf)

Pesymistyczna złożoność czasowa $O(|V||E|)$.

Każdy znaleziony ujemny cykl zostanie zlikwidowany, przez przesłanie przez niego tyle przepływu na ile pozwala nam przepustowość. Jako że poruszanie się po ujemnym cyklu zmniejsza nam koszt, całkowity koszt także zmaleje. Przesyłanie przepływu w zamkniętym cyklu nie zmieni nam całkowitego przepływu, więc zostanie on maksymalny. Gdy algorytm nie będzie w stanie znaleźć ujemnego cyklu, oznacza to, że minimalny koszt został osiągnięty.

Pesymistyczna złożoność czasowa $O(C|V||E|)$, gdzie C to ilość usunięć ujemnego cyklu.

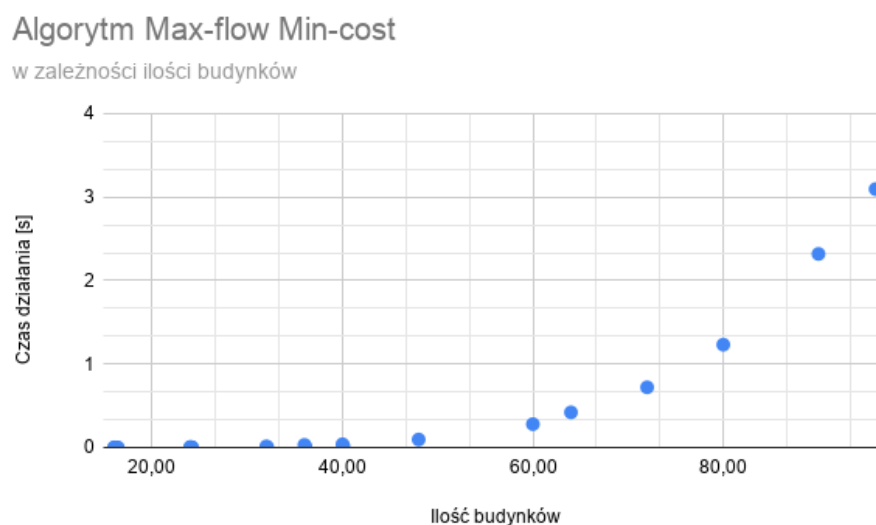
Ze znalezionego maksymalnego przepływu o minimalnym koszcie, wyciągamy krawędzie między domkami, a kopalniami z przepływem większym od zera. Wszystkie takie krawędzie definiują nam przydzielenie krasnoludkom pracy w kopalni, o największej liczbie parowań, największej liczbie wydobywanych surowców i najkrótszej łącznej trasie. Poprawność tego algorytmu opisana jest w publikacji "Finding minimum-cost circulations by canceling negative cycles" autorów algorytmu (<https://dl.acm.org/doi/10.1145/76359.76368>)

5.4 Uzasadnienie

Wykorzystanie algorytmu Edmondsa-Karpa podyktowała prostota implementacji, która gwarantuje znalezienie największego przepływu, jako że posiada warunek stopu dla liczb całkowitych. Algorytm Bellmana-Forda jako jeden z nielicznych pozwala nam na znalezienie ujemnego cyklu, który potrzebny jest nam do algorytmu Cycle Canceling. Użycie jego wersji zmodyfikowanej przez Edwarda F. Moore'a było sposobem przyspieszenia całkowitego czasu działania.

5.5 Pomiary

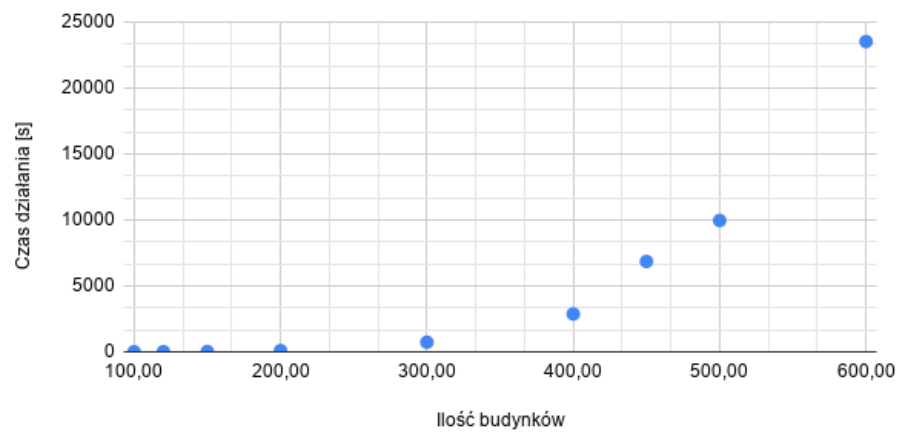
Przeprowadziliśmy testy na wygenerowanych losowo miastach o różnej ilości budynków, wymiarach miasta, proporcji domków do kopalni i ilościach dostępnych materiałów. Średni czas wykonania algorytmu dla miast mniejszych niż 80 budynków wynosił poniżej sekundy.



Dla miast większych niż 80 budynków czas zaczynał rosnąć szybciej. Dla 400 budynków czas wynosił 47 minut, a dla 600 budynków wynosił około 6.5 godziny. Poniżej 200 budynków średni czas nie przekraczał 2 minut.

Algorytm Max-flow Min-cost

w zależności ilości budynków



Poniżej wykres ze wszystkimi pomiarami w skali logarytmicznej. Kształt odpowiada sześcienniej złożoności czasowej, która wynika z implementacji dwóch algorytmów o złożoności $O(|V||E|^2) + O(C|V||E|)$

Algorytm Max-flow Min-cost

w zależności ilości budynków, skala logarytmiczna

